

Learning FRAME Models Using CNN Filters for Knowledge Visualization

Yang Lu, Song-Chun Zhu, Ying Nian Wu

Department of Statistics, University of California, Los Angeles, USA

Abstract

The convolutional neural network (ConvNet or CNN) has proven to be very successful in many tasks such as those in computer vision. Recently there has been growing interest in visualizing the knowledge discriminatively learned by CNNs, by generating images based on CNN features. This paper is a contribution towards this theme of research on knowledge visualization via image generation. Specifically, we propose to learn the generative FRAME (Filters, Random field, And Maximum Entropy) model using the highly expressive filters pre-learned by the CNN at the convolutional layers. We show that the learning algorithm can generate vivid images, and we explain that each learned model corresponds to a CNN unit at a layer above the layer of filters employed by the model. Such a CNN-FRAME generative model is different from existing visualization methods, and the proposed learning method enables us to add CNN nodes by learning from small numbers of training examples in a generative fashion.

1. Introduction

Ever since the recent breakthrough by the convolutional neural network (ConvNet or CNN) (Krizhevsky, Sutskever, and Hinton, 2012; LeCun et al., 1998) on the ImageNet dataset (Deng et al., 2009), researchers have been interested in understanding the knowledge discriminatively learned by CNN. One way to probe the mind of a CNN is to visualize the knowledge of the network by generating images using the CNN features at different layers. To this end, one can either generate images by maximizing the responses of CNN units (Simonyan, Vedaldi, and Zisserman, 2015), or generating images to match the CNN features of an input image (Gatys, Ecker, and Bethge, 2015).

In this article, we propose to visualize the knowledge of CNN features by learning generative FRAME (Filters, Random field, And Maximum Entropy) models (Zhu, Wu, and Mumford, 1997; Xie et al., 2015a) using the highly sophisticated and nonlinear filters pre-learned by CNN at the convolutional layers. A FRAME model is a random field model defined on the image space. The model is generative in the sense that images can be generated by sampling from the probability distribution defined by the model. The probability distribution is the maximum entropy distribution that reproduces the statistical properties of filter responses in the

observed images. Being of the maximum entropy, the distribution is the most random distribution that matches the observed statistical properties of filter responses, so that images sampled from this distribution can be considered typical images that share the statistical properties of the observed images.

There are two versions of FRAME models in the literature. The original version is a stationary model developed for modeling texture patterns (Zhu, Wu, and Mumford, 1997). The more recent version is a non-stationary extension designed to represent object patterns (Xie et al., 2015a). Both versions of the FRAME models can be sparsified by selecting a subset of filters from a given dictionary.

The filters used in the FRAME model are the oriented and elongated Gabor filters at different scales, as well as the isotropic Difference of Gaussian (DoG) filters of different sizes. These are linear filters that capture simple local image features such as edges and blobs. With the emergence of the more powerful nonlinear filters learned by CNN at various convolutional layers, it is only natural to replace the linear filters in the original FRAME models by the CNN filters in the hope of learning more expressive models.

Incorporating CNN filters into the FRAME model is not a mere utilitarian exploit. It is actually a seamless meshing between the FRAME model and the CNN model. The original FRAME model has an energy function that involves a layer of linear filtering followed by a layer of pointwise nonlinear transformation. It is natural to follow the deep learning philosophy to add alternative layers of linear filtering and nonlinear transformation to have a generative model that directly corresponds to a CNN. More importantly, the learned FRAME model using CNN filters corresponds to a new CNN unit at the layer directly above the layer of CNN filters employed by the FRAME model. In particular, the non-stationary FRAME becomes a single CNN node at a specific position where the object appears, whereas the stationary FRAME becomes a special type of convolutional unit. Therefore, the learned FRAME model can be viewed as a generative version of CNN unit.

In this article, we focus on knowledge visualization via image generation. We show that by learning FRAME models with CNN filters, we can generate vivid object and texture patterns. This enables us to learn new CNN units from a small number of training images in a generative fashion.

The main purpose of this paper is to establish the conceptual correspondence between the generative FRAME model and the discriminative CNN, thus providing a generative perspective for CNN. Such a perspective is much needed because it enables us to visualize the knowledge learned by CNN in a principled way. More importantly, it may eventually lead to unsupervised learning of knowledge in a generative fashion without the need for image labeling.

2. Past work

Recently there have been many interesting papers on visualizing CNN nodes, such as deconvolutional networks (Zeiler and Fergus, 2014), score maximization (Simonyan, Vedaldi, and Zisserman, 2015), and the recent artful work of Google deep dream (<http://deepdreamgenerator.com/>) and painting style (Gatys, Ecker, and Bethge, 2015). Our work is different from these previous methods in that we learn a rigorously defined generative model from training images, and the learned models correspond to new CNN units. This work is a continuation of the recent work on generative CNN (Dai, Lu, and Wu, 2015).

There have also been recent papers on generative models based on supervised image generation (Dosovitskiy, Springenberg, and Brox, 2015), variational auto-encoders (Hinton et al., 1995; Kingma and Welling, 2014; Rezende, Mohamed, and Wierstra, 2014; Mnih and Gregor, 2014; Kulka-rni et al., 2015; Gregor et al., 2015), and adversarial networks (Denton et al., 2015). Each of these papers learns a top-down multi-layer model for image generation, and the parameters of the top-down generation model are completely separated from the parameters of the bottom-up recognition model. Our work seeks to learn a generative model based on the knowledge learned by the bottom-up recognition model, i.e., the image generation model and the image recognition model share the same set of parameters.

3. FRAME models based on linear filters

This section reviews the background on the FRAME models based on linear filters.

Let $\mathbf{I}$ be an image defined on a square (or rectangular) domain $\mathcal{D}$. Let $\{F_k, k = 1, \dots, K\}$ be a bank of linear filters, such as elongate and oriented Gabor filters at different scales, as well as isotropic Difference of Gaussian (DoG) filters of different sizes. Let $F_k * \mathbf{I}$ be the filtered image or feature map, and $[F_k * \mathbf{I}](x)$ be the filter response at position x (x is a two-dimensional coordinate). A linear filter F_k can be written as a two-dimensional function $F_k(x)$, so that $[F_k * \mathbf{I}](y) = F_k(x)\mathbf{I}(y+x)$, which is a translation invariant linear operation.

The original FRAME model (Zhu, Wu, and Mumford, 1997) for texture patterns is a stationary or spatially homogeneous Markov random field or Gibbs distribution of the following form:

$$p(\mathbf{I}; \lambda) = \frac{1}{Z(\lambda)} \exp \left(\sum_{k=1}^K \sum_{x \in \mathcal{D}} \lambda_k ([F_k * \mathbf{I}](x)) \right), \quad (1)$$

where $\lambda_k()$ is a nonlinear function to be estimated from the training images, $\lambda = (\lambda_k(), k = 1, \dots, K)$, and $Z(\lambda)$ is the

normalizing constant to make $p(\mathbf{I}; w)$ integrate to 1. In the original paper of Zhu, Wu, and Mumford (1997), each $\lambda_k()$ is discretized and estimated as a step function, i.e., $\lambda_k(r) = \sum_{b=1}^B w_{k,b} h_b(r)$, where $b \in \{1, \dots, B\}$ indexes the equally spaced bins of discretization, and $h_b(r) = 1$ if r is in bin b , and 0 otherwise, i.e., $h() = (h_b(), b = 1, \dots, B)$ is a 1-hot indicator vector, and $\sum_x h([F_k * \mathbf{I}](x))$ is the marginal histogram of filter map $F_k * \mathbf{I}$. The spatially pooled marginal histograms are the sufficient statistics of model (1).

Model (1) is stationary because the function $\lambda_k()$ does not depend on position x . This stationary model is used to model texture patterns. In model (1), the energy function $U(\mathbf{I}; \lambda) = -\sum_k \sum_x \lambda_k([F_k * \mathbf{I}](x))$ involves a layer of linear filtering by $\{F_k\}$, followed by a layer of pointwise nonlinear transformation by $\{\lambda_k()\}$. Repeating this pattern recursively (while also adding local max pooling and subsampling) will lead to a generative version of CNN.

The non-stationary or spatially inhomogeneous FRAME model for object patterns (Xie et al., 2015a) is of the following form:

$$p(\mathbf{I}; \lambda) = \frac{1}{Z(\lambda)} \exp \left(\sum_{k=1}^K \sum_{x \in \mathcal{D}} \lambda_{k,x}([F_k * \mathbf{I}](x)) \right) q(\mathbf{I}), \quad (2)$$

where the function $\lambda_{k,x}()$ depends on position x , and $\lambda = (\lambda_{k,x}(), \forall k, x)$. Again $Z(\lambda)$ is the normalizing constant. The model is non-stationary because $\lambda_{k,x}()$ depends on position x . It is impractical to estimate $\lambda_{k,x}()$ as a step function at each x , so $\lambda_{k,x}()$ is parametrized as a one-parameter function

$$\lambda_{k,x}(r) = w_{k,x} h(r), \quad (3)$$

where $h()$ is a pre-specified rectification function, and $w = (w_{k,x}, \forall k, x)$ are the unknown parameters to be estimated. In the paper of Xie et al. (2015a), they use $h(r) = |r|$ for full wave rectification. One can also use rectified linear unit $h(r) = \max(0, r)$ (Krizhevsky, Sutskever, and Hinton, 2012) for half wave rectification, which can be considered an elaborate two-bin discretization. $q(\mathbf{I})$ is a reference distribution, such as the Gaussian white noise model

$$q(\mathbf{I}) = \frac{1}{(2\pi\sigma^2)^{|\mathcal{D}|/2}} \exp \left(-\frac{1}{2\sigma^2} \|\mathbf{I}\|^2 \right), \quad (4)$$

where $|\mathcal{D}|$ counts the number of pixels in the image domain $\mathcal{D}$.

In the original FRAME model (1), $q(\mathbf{I})$ is assumed to be a uniform measure. In model (2), we can also absorb $q(\mathbf{I})$, in particular, the $\frac{1}{2\sigma^2} \|\mathbf{I}\|^2$ term, into the energy function, so that the model is again defined relative to a uniform measure as in the original FRAME model (1). We make $q(\mathbf{I})$ explicit here because we shall specify the parameter σ^2 instead of learning it, and use $q(\mathbf{I})$ as the null model for the background. In models (2) and (3), $(w_{k,x}, \forall x, k)$ can be considered a second-layer linear filter on top of the first layer filters $\{F_k\}$ rectified by $h()$.

Both models (1) and (2) can be sparsified. Model (1) can be sparsified by selecting a small set of filters F_k using the filter pursuit procedure (Zhu, Wu, and Mumford, 1997).

Model (2) can be sparsified by selecting a small number of filters F_k and positions x , so that only a small number of $w_{k,x}$ are non-zero. The sparsification can be achieved by a shared matching pursuit method (Xie et al., 2015a) or a generative boosting method (Xie et al., 2015b).

4. FRAME models based on CNN filters

Instead of using linear filters, we can use the filters at various convolutional layers of a CNN. Suppose there exists a bank of filters $\{F_k, k = 1, \dots, K\}$ (e.g., $K = 512$) at a certain convolutional layer of a pre-learned CNN. For an image $\mathbf{I}$ defined on the square image domain $\mathcal{D}$, let $F_k * \mathbf{I}$ be the feature map of filter F_k , and let $[F_k * \mathbf{I}](x)$ be the filter response of $\mathbf{I}$ to F_k at position x (again x is a two-dimensional coordinate). We assume that $[F_k * \mathbf{I}](x)$ is the response obtained after applying the rectified linear transformation $h(r) = \max(0, r)$. Then the non-stationary FRAME model becomes

$$p(\mathbf{I}; w) = \frac{1}{Z(w)} \exp \left(\sum_{k=1}^K \sum_{x \in \mathcal{D}} w_{k,x} [F_k * \mathbf{I}](x) \right) q(\mathbf{I}), \quad (5)$$

where $q(\mathbf{I})$ is again the Gaussian white noise model (4), and $w = (w_{k,x}, \forall k, x)$ are the unknown parameters to be learned from the training data. $Z(w)$ is the normalizing constant. Model (5) shares the same form as model (2) with linear filters, except that the rectification function $h(\cdot)$ in model (2) is already absorbed in the CNN filters $\{F_k\}$ in model (5) with $h(r) = \max(0, r)$. We shall use model (5) for generating object patterns.

The stationary FRAME model is of the following form:

$$p(\mathbf{I}; w) = \frac{1}{Z(w)} \exp \left(\sum_{k=1}^K \sum_{x \in \mathcal{D}} w_k [F_k * \mathbf{I}](x) \right) q(\mathbf{I}), \quad (6)$$

which is almost the same as model (5) except that w_k is the same across x , i.e. $w = (w_k, \forall k)$. We shall use model (6) for generating texture patterns.

Again, both models (5) and (6) can be sparsified, either by forward selection such as filter pursuit (Zhu, Wu, and Mumford, 1997) or generative boosting (Xie et al., 2015b), or by backward elimination.

5. Learning and sampling algorithm

The basic learning algorithm for object model estimates the unknown parameters w from a set of aligned training images $\{\mathbf{I}_m, m = 1, \dots, M\}$ that come from the same object category, where M is the total number of training images. In the basic learning algorithm, the weight parameters w can be estimated by maximizing the log-likelihood function

$$L(w) = \frac{1}{M} \sum_{m=1}^M \log p(\mathbf{I}_m; w), \quad (7)$$

where $p(\mathbf{I}; w)$ is defined by (5). $L(w)$ is a concave function. The first derivatives of $L(w)$ are

$$\frac{\partial L(w)}{\partial w_{k,x}} = \frac{1}{M} \sum_{m=1}^M [F_k * \mathbf{I}_m](x) - \mathbb{E}_w([F_k * \mathbf{I}](x)), \quad (8)$$

where $\mathbb{E}_w$ denotes expectation with respect to $p(\mathbf{I}; w)$. The expectation can be approximated by Monte Carlo integration. The second derivative of $L(w)$ is the variance-covariance matrix of $([F_k * \mathbf{I}](x), \forall k, x)$. w can be computed by a stochastic gradient ascent algorithm (Younes, 1999):

$$w_{k,x}^{(t+1)} = w_{k,x}^{(t)} + \gamma \left(\frac{1}{M} \sum_{m=1}^M [F_k * \mathbf{I}_m](x) - \frac{1}{\tilde{M}} \sum_{m=1}^{\tilde{M}} [F_k * \tilde{\mathbf{I}}_m](x) \right) \quad (9)$$

for every $k \in \{1, \dots, K\}$ and $x \in \mathcal{D}$, where γ is the learning rate, and $\{\tilde{\mathbf{I}}_m\}$ are the synthesized images sampled from $p(\mathbf{I}; w^{(t)})$ using MCMC. $\tilde{M}$ is the total number of independent parallel Markov chains that sample from $p(\mathbf{I}; w^{(t)})$. The learning rate γ can be made inversely proportional to the observed variance of $\{[F_k * \mathbf{I}_m](x), \forall m\}$, as well as being inversely proportional to the iteration t as in stochastic approximation.

In order to sample from $p(\mathbf{I}; w)$, we adopt the Langevin dynamics. Writing the energy function

$$U(\mathbf{I}, w) = - \sum_{k=1}^K \sum_{x \in \mathcal{D}} w_{k,x} [F_k * \mathbf{I}](x) + \frac{1}{2\sigma^2} \|\mathbf{I}\|^2. \quad (10)$$

The Langevin dynamics iterates

$$\mathbf{I}_{\tau+1} = \mathbf{I}_\tau - \frac{\epsilon^2}{2} U'(\mathbf{I}_\tau, w) + \epsilon Z_\tau, \quad (11)$$

where $U'(\mathbf{I}, w) = \partial U(\mathbf{I}, w) / \partial \mathbf{I}$. This gradient can be computed by back-propagation. In (11), ϵ is a small step-size, and $Z_\tau \sim \mathcal{N}(0, \mathbf{1})$, independently across τ , where the bold font $\mathbf{1}$ is the identity matrix, i.e., Z_t is a Gaussian white noise image whose pixel values follow $\mathcal{N}(0, 1)$ independently. Here we use τ to denote the time steps of the Langevin sampling process, because t is used for the time steps of the learning process. The Langevin sampling process is an inner loop within the learning process. Between every two consecutive updates of w in the learning process, we run a finite number of iterations of the Langevin dynamics starting from the images generated by the previous iteration of the learning algorithm, a scheme called “warm start” in the literature. The Langevin equation was also adopted by Zhu and Mumford (1997), who called the corresponding gradient descent algorithm the Gibbs reaction and diffusion equation (GRADE). One can also replace Langevin dynamics by Hamiltonian Monte Carlo (Neal, 2011).

Algorithm 1 describes the details of the learning and sampling algorithm. Algorithm 1 embodies the principle of “analysis by synthesis”, i.e., we generate synthesized images from the current model, and then update the model parameters based on the difference between the synthesized images and the observed images. If we consider the synthesized images as from the dream of the current model (following the metaphor used by Google deep dream), then the learning algorithm is to make the dream come true.

From the MCMC perspective, Algorithm 1 runs non-stationary parallel Markov chains that sample from a Gibbs

Algorithm 1 Learning and sampling algorithm

Input:

- (1) training images $\{\mathbf{I}_m, m = 1, \dots, M\}$
- (2) a filter bank $\{F_k, k = 1, \dots, K\}$
- (3) number of synthesized images $\tilde{M}$
- (4) number of Langevin steps L

Output:

- (1) estimated parameters $w = (w_{k,x}, \forall k, x)$
- (2) synthesized images $\{\tilde{\mathbf{I}}_m, m = 1, \dots, \tilde{M}\}$

1: Calculate observed statistics:

$$H_{k,x}^{\text{obs}} \leftarrow \frac{1}{M} \sum_{m=1}^M [F_k * \mathbf{I}_m](x), \forall k, x.$$

2: Let $t \leftarrow 0$, initialize $w_{k,x}^{(0)} \leftarrow 0, \forall k, x$.

3: Initialize $\tilde{\mathbf{I}}_m \leftarrow 0$, for $m = 1, \dots, \tilde{M}$.

4: **repeat**

5: For each m , run L steps of Langevin dynamics to update $\tilde{\mathbf{I}}_m$, i.e., starting from the current $\tilde{\mathbf{I}}_m$, each step updates $\tilde{\mathbf{I}}_m \leftarrow \tilde{\mathbf{I}}_m - \frac{\epsilon^2}{2} U'(\tilde{\mathbf{I}}_m, w^{(t)}) + \epsilon Z$, where $Z \sim \mathcal{N}(0, \mathbf{I})$.

6: Calculate synthesized statistics:

$$H_{k,x}^{\text{syn}} \leftarrow \frac{1}{\tilde{M}} \sum_{m=1}^{\tilde{M}} [F_k * \tilde{\mathbf{I}}_m](x), \forall k, x.$$

7: Update $w_{k,x}^{(t+1)} \leftarrow w_{k,x}^{(t)} + \gamma(H_{k,x}^{\text{obs}} - H_{k,x}^{\text{syn}}), \forall k, x$.

8: Let $t \leftarrow t + 1$

9: **until** $\sum_{k,x} |H_{k,x}^{\text{obs}} - H_{k,x}^{\text{syn}}| \leq \epsilon$

distribution with a changing energy landscape, like in simulated annealing or tempering. This may help the chains to avoid the trapping of local modes. We can also use “cold start” scheme by initializing Langevin dynamics from white noise images in each learning iteration and allowing the dynamics enough time to relax.

For learning stationary FRAME (6), usually $M = 1$, i.e., we observe one texture image, and we update the parameters

$$w_k^{(t+1)} = w_k^{(t)} + \frac{\gamma}{|\mathcal{D}|} \left(\frac{1}{M} \sum_{m=1}^M \sum_{x \in \mathcal{D}} [F_k * \mathbf{I}_m](x) - \frac{1}{\tilde{M}} \sum_{m=1}^{\tilde{M}} \sum_{x \in \mathcal{D}} [F_k * \tilde{\mathbf{I}}_m](x) \right) \quad (12)$$

for every $k \in \{1, \dots, K\}$, where there is a spatial pooling across positions $x \in \mathcal{D}$. The sampling is again accomplished by Langevin dynamics. The learning and sampling algorithm for the stationary model (6) only involves minor modifications of Algorithm 1.

6. Maximum entropy justification

The FRAME model (5) can be justified by the maximum entropy or minimum divergence principle. Suppose the true distribution that generates the observed images $\{\mathbf{I}_m\}$ is $f(\mathbf{I})$. Let w^* solve the population version of the maximum likelihood equation:

$$\mathbb{E}_w([F_k * \mathbf{I}](x)) = \mathbb{E}_f([F_k * \mathbf{I}](x)), \forall k, x. \quad (13)$$

Let Ω be the set of all the feasible distributions p that share the statistical properties of f as captured by $\{F_k\}$:

$$\Omega = \{p : \mathbb{E}_p([F_k * \mathbf{I}](x)) = \mathbb{E}_f([F_k * \mathbf{I}](x)) \forall k, x\}. \quad (14)$$

Then it can be shown that among all $p \in \Omega$, $p(\mathbf{I}; w^*)$ achieves the minimum of $\text{KL}(p||q)$, i.e., the Kullback-Leibler divergence from p to q (Della Pietra, Della Pietra, and Lafferty, 1997). Thus $p(\mathbf{I}; w^*)$ can be considered the projection of q onto Ω , or the minimal modification of the reference distribution q to match the statistical properties of the true distribution f . In the special case where q is a uniform distribution, $p(\mathbf{I}; w^*)$ achieves the maximum entropy among all distributions in Ω . For Gaussian white noise q , as mentioned before, we can absorb the $\frac{\|\mathbf{I}\|^2}{2\sigma^2}$ term into the energy function as in (10), so model (5) can be written relative to a uniform measure with $\|\mathbf{I}\|^2$ as an additional feature. The maximum entropy interpretation thus still holds if we opt to estimate σ^2 from the data.

7. Julesz ensemble justification

The learning algorithm seeks to match statistics of the synthesized images to those of the observed images, as indicated by (9) and (12), where the difference between the observed statistics and the synthesized statistics drives the update of the parameters. If the algorithm converges, and if the number of the synthesized images $\tilde{M}$ is large in the case of object patterns or if the image domain $\mathcal{D}$ is large in the case of texture patterns, then the synthesized statistics should match the observed statistics. Assume $q(\mathbf{I})$ to be uniform distribution for now. We can consider the following ensemble in the case of object patterns:

$$\begin{aligned} \mathcal{J} &= \left\{ (\tilde{\mathbf{I}}_m, m = 1, \dots, \tilde{M}) : \frac{1}{\tilde{M}} \sum_{m=1}^{\tilde{M}} [F_k * \tilde{\mathbf{I}}_m](x) \right. \\ &= \left. \frac{1}{M} \sum_{m=1}^M [F_k * \mathbf{I}_m](x), \forall k, x \right\}. \end{aligned} \quad (15)$$

Consider the uniform distribution over $\mathcal{J}$. Then as $\tilde{M} \rightarrow \infty$, the marginal distribution of any $\tilde{\mathbf{I}}_m$ is given by model (5) with w being estimated by maximum likelihood. Conversely, model (5) puts uniform distribution on $\mathcal{J}$ if $\tilde{\mathbf{I}}_m$ are independent samples from model (5) and if $\tilde{M} \rightarrow \infty$.

As for texture model, we can take $\tilde{M} = 1$, but let the image size go to ∞ . First fix the square domain $\mathcal{D}$. Then embed it at the center of a larger square domain $\overline{\mathcal{D}}$. Consider the ensemble of images defined on $\overline{\mathcal{D}}$:

$$\begin{aligned} \mathcal{J} &= \left\{ \tilde{\mathbf{I}} : \frac{1}{|\overline{\mathcal{D}}|} \sum_{x \in \overline{\mathcal{D}}} [F_k * \tilde{\mathbf{I}}](x) \right. \\ &= \left. \frac{1}{|\mathcal{D}|} \frac{1}{M} \sum_{m=1}^M \sum_{x \in \mathcal{D}} [F_k * \mathbf{I}_m](x), \forall k \right\}. \end{aligned} \quad (16)$$

Then under the uniform distribution on $\mathcal{J}$, as $|\overline{\mathcal{D}}| \rightarrow \infty$, the distribution of $\tilde{\mathbf{I}}$ restricted to $\mathcal{D}$ is given by model (6). Conversely, model (6) defined on $\overline{\mathcal{D}}$ puts uniform distribution on $\mathcal{J}$ as $|\overline{\mathcal{D}}| \rightarrow \infty$.

The ensemble $\mathcal{J}$ is called Julesz ensemble by Wu, Zhu, and Liu (2000), because Julesz was the first to pose the question as to what statistics define a texture pattern (Julesz, 1962). The averaging across images in equation (15) enables us to exchange the parts of the observed images to generate new object images. The spatial averaging in equation (16) enables us to shuffle the local patterns in the observed image to generate a new texture image. That is, the averaging operations lead to exchangeability.

For object patterns, define the discrepancy

$$\Delta_{k,x} = \frac{1}{M} \sum_{m=1}^{\tilde{M}} [F_k * \tilde{\mathbf{I}}_m](x) - \frac{1}{M} \sum_{m=1}^M [F_k * \mathbf{I}_m](x). \quad (17)$$

One can sample from the uniform distribution on $\mathcal{J}$ in (15) by running a simulated annealing algorithm that samples from $p(\tilde{\mathbf{I}}_m, m = 1, \dots, \tilde{M}) \propto \exp(-\sum_{k,x} \Delta_{k,x}^2/T)$ by Langevin dynamics while gradually lowering the temperature T , or simply by gradient descent as in Gatys, Ecker, and Bethge (2015) by assuming $T = 0$. The sampling algorithm is very similar to Algorithm 1. One can use a similar method to sample from the uniform distribution over $\mathcal{J}$ in (16). Such a scheme was used by Zhu, Liu, and Wu (2000) for texture synthesis.

In the above discussion, we assume $q(\mathbf{I})$ to be the uniform distribution. If $q(\mathbf{I})$ is Gaussian, we only need to add the feature $\|\mathbf{I}\|^2$ to the pool of features to be matched. The above results still hold.

The Julesz ensemble perspective connects statistics matching and the FRAME models, thus providing another justification for these models in addition to the maximum entropy principle.

8. Generative CNN units

On top of the convolutional layer of filters $\{F_k, k = 1, \dots, K\}$, we can build another layer of filters $\{\mathbf{F}_j, j = 1, \dots, J\}$ (with $\mathbf{F}$ in bold font, and indexed by j), so that

$$[\mathbf{F}_j * \mathbf{I}](y) = h \left(\sum_{k,x} w_{k,x}^{(j)} [F_k * \mathbf{I}](y+x) + b_j \right), \quad (18)$$

where $h(\cdot)$ is a rectification function such as the rectified linear unit $h(r) = \max(0, r)$, and where the bias term b_j is related to $-\log Z(w)$. For simplicity, we ignore the layers of local max pooling and sub-sampling.

Model (5) corresponds to a single filter in $\{\mathbf{F}_j\}$ at a particular position y (e.g., the origin $y = 0$) where we assume that the object appears. The weights $(w_{k,x}^{(j)})$ can be learned by fitting model (5) using Algorithm 1, which enables us to add a CNN node in a generative fashion.

The log-likelihood ratio of object model $p(\mathbf{I}; w)$ versus background $q(\mathbf{I})$ is $\log(p(\mathbf{I}; w)/q(\mathbf{I})) = \sum_k \sum_x w_{k,x} [F_k * \mathbf{I}](x) - \log Z(w)$. It can be used as a score for detecting the object versus the background. If the score is below a threshold, no object is detected, and the score is rectified to 0. The rectified linear unit $h(\cdot)$ in $\mathbf{F}_j$ in (18) accounts for the fact that at any position y , the object either



Figure 1: Generating object patterns. For each category, the first row displays 4 of the training images, and the second row displays generated images.

appears or not. More formally, consider a mixture model $p(\mathbf{I}) = \alpha p(\mathbf{I}; w) + (1 - \alpha)q(\mathbf{I})$, where α is the frequency that the object is activated, and $1 - \alpha$ is the frequency of background. $\log(p(\mathbf{I})/q(\mathbf{I})) = \log(1 + \exp(\sum_k \sum_x w_{k,x}(F_k * \mathbf{I})(x) - \log Z(w) + \log(\alpha/(1 - \alpha)) + \log(1 - \alpha))$. We can approximate the soft max function $\log(1 + e^r)$ by the hard max function $\max(0, r)$. Thus we can identify the bias term as $b = \log(\alpha/(1 - \alpha)) - \log Z(w)$, and the rectified linear unit models a mixture of “on” and “off” of an object pattern.

Model (5) is used to model images where the objects are aligned and are from the same category. For images of non-aligned objects from multiple categories, we can extend model (5) to a convolutional version with multiple filters

$$p(\mathbf{I}; w) = \frac{1}{Z(w)} \exp \left(\sum_{j=1}^J \sum_{x \in \mathcal{D}} [\mathbf{F}_j * \mathbf{I}](x) \right) q(\mathbf{I}), \quad (19)$$

where $\{\mathbf{F}_j\}$ are defined by (18). This model is a product of experts model (Hinton, 2002), where each $[\mathbf{F}_j * \mathbf{I}](x)$ is an expert about a mixture of an activation or not of an object of type j at position x . We can introduce an explicit latent binary switch variable of activation or not for each expert. We call model (19) with (18) the generative CNN model. The model can also be considered a dense version of the And-Or model advocated by Zhu and Mumford (2006), where the binary switch of each expert corresponds to an Or-node, and the product corresponds to an And-node.

The stationary model (6) corresponds to a special case of generative CNN model (19) with (18), where there is only one j , and $[\mathbf{F} * \mathbf{I}](x) = \sum_{k=1}^K w_k [F_k * \mathbf{I}](x)$, which is a special case of (18) without rectification. It is a singleton filter that combines lower layer filter responses at the same position.

More importantly, due to the recursive nature of CNN, if the weight parameters w_k of the stationary model (6) are absorbed into the filters F_k by multiplying the weight and bias parameters of each F_k by w_k , then the stationary model becomes the generative CNN model (19) except that the top-layer filters $\{\mathbf{F}_j\}$ are replaced by the lower layer filters $\{F_k\}$. The learning of the stationary model (6) is a simplified version of the learning of the generative CNN model (19) where there is only one multiplicative parameter w_k for each filter F_k . The learning of the stationary model (6) is more unsupervised and more indicative of the expressiveness of the CNN features than the learning of the non-stationary model (5) because the former does not require alignment.

Suppose we observe $\{\mathbf{I}_m, m = 1, \dots, M\}$ from the generative CNN model (19) with (18). Let $L(w) = \frac{1}{M} \sum_{m=1}^M \log p(\mathbf{I}_m; w)$ be the log-likelihood where $p(\mathbf{I}; w)$ is defined by (19) and (18), then

$$\begin{aligned} \frac{\partial L(w)}{\partial w_{k,x}^{(j)}} &= \frac{1}{M} \sum_{m=1}^M \sum_{y \in \mathcal{D}} \delta_{j,y}(\mathbf{I}_m) [F_k * \mathbf{I}_m](y + x) \\ &\quad - \mathbb{E}_w \left[\sum_{y \in \mathcal{D}} \delta_{j,y}(\mathbf{I}) [F_k * \mathbf{I}](y + x) \right], \end{aligned} \quad (20)$$



Figure 2: Generating texture patterns. For each category, the first image is the training image, and the next two images are generated images.



Figure 3: Generating hybrid object patterns. For each experiment, the first row displays 4 of the training images, and the second row displays generated images.

where

$$\delta_{j,y}(\mathbf{I}) = h' \left(\sum_{k,x} w_{k,x}^{(j)} [F_k * \mathbf{I}](y+x) + b_j \right) \quad (21)$$

is a binary on/off detector of object j at position y on image $\mathbf{I}$. The binary detector corresponds to the binary switch variable in each expert of the product of expert model (19). Specifically, for $h(r) = \max(0, r)$, $h'(r) = 0$ if $r \leq 0$, and $h'(r) = 1$ if $r > 0$. The gradient (20) can be called generative gradient, because it can be approximated by generating images from the current model. The partial derivatives with respect to the bias parameters can be similarly derived and they are to match the numbers of occurrences of the objects. The gradient (20) admits an EM-type interpretation which is typical in unsupervised learning algorithms that involve hidden variables. Specifically, $\delta_{j,y}()$ infers the binary switch variable in each expert in model (19), and it can be considered a hard-decision E-step. With the latent switch variables inferred, the parameters are updated in the same way as in the supervised learning algorithm, such as the one based on (8), which can be considered the M-step. Thus model (19) is an energy-based model (LeCun et al., 2006) with implicit binary latent variables. As to unsupervised learning, a similar detection and re-learning scheme was used by Hong et al. (2014) to learn codebooks of active basis models (Wu et al., 2010).

To the filters $\{\mathbf{F}_j\}$ (bold font) in (18), $\{F_k\}$ (not bold) form a lower layer of base filters. We have been assuming that the base filters $\{F_k\}$ in (18) are already pre-learned. If they are not pre-learned, we can also learn the weight parameters that define $\{F_k\}$ via back-propagation. Thus, the



Figure 4: Generating hybrid texture patterns. The first two images are training images, and the last two images are generated images.

parameter w in model (19) can be interpreted more broadly as multi-layer connection weights that define all the layers of filters. The gradient of the log-likelihood is

$$\begin{aligned} \frac{\partial L(w)}{\partial w} = & \frac{1}{M} \sum_{m=1}^M \sum_{j=1}^J \sum_{x \in \mathcal{D}} \frac{\partial}{\partial w} [\mathbf{F}_j * \mathbf{I}_m](x) \\ & - \mathbb{E}_w \left[\sum_{j=1}^J \sum_{x \in \mathcal{D}} \frac{\partial}{\partial w} [\mathbf{F}_j * \mathbf{I}](x) \right], \end{aligned} \quad (22)$$

where $\partial[\mathbf{F}_j * \mathbf{I}](x)/\partial w$ involves multiple layers of binary detectors inferring the implicit hidden switch variables at multiple layers. The resulting algorithm also requires partial derivative $\partial[\mathbf{F}_j * \mathbf{I}](x)/\partial \mathbf{I}$ for Langevin sampling, which can be considered a recurrent generative model driven by the binary switches at multiple layers. Both $\partial[\mathbf{F}_j * \mathbf{I}](x)/\partial w$ and $\partial[\mathbf{F}_j * \mathbf{I}](x)/\partial \mathbf{I}$ are readily available via back-propagation. See Hinton et al. (2006); Ngiam et al. (2011) for earlier work along this direction. See also Dai, Lu, and Wu (2015) for generative gradient of CNN.

Finally, we can also learn a FRAME model based on the features at the top fully connected layer,

$$p(\mathbf{I}; W) = \frac{1}{Z(W)} \exp \left(\sum_{i=1}^N W_i [\Psi_i * \mathbf{I}] \right) q(\mathbf{I}), \quad (23)$$

where Ψ_i is the i -th feature at the top fully connected layer, N is the total number of features at this layer (e.g., $N = 4096$), and W_i are the parameters, $W = (W_i, \forall i)$. Ψ_i can still be viewed as a filter whose filter map is 1×1 . Suppose there are a number of image categories, and suppose we learn a model (23) for each image category with a category-specific W . Also suppose we are given the prior frequency of each category. A simple exercise of Bayes rule then gives us the soft-max classification rule for the posterior probability of category given image, which is the discriminative CNN.

9. Image generation experiments

In our experiments, we use VGG filters (Simonyan and Zisserman, 2015), and we use the Matlab code of MatConvNet (Vedaldi and Lenc, 2014).

Experiment 1: generating object patterns. We learn the non-stationary FRAME models (5) from images of aligned objects. The images are collected from the internet. For each category, the number of training images is around 10. We use



Figure 5: Generating scene patterns. Learning filters of Conv4 without requiring object bounding boxes or image alignment. The first image is the training image, and the rest 4 images are generated images.

$\tilde{M} = 16$ parallel chains for Langevin sampling. The number of Langevin iterations between every two consecutive updates of the parameters is $L = 100$. Fig. 1 shows some experiments using filters from the 3rd convolutional layer of VGG. For each experiment, the first row displays 4 of the training images, and the second row displays 4 of the synthesized images generated by Algorithm 1.

Experiment 2: generating texture patterns. We learn the stationary FRAME models (6) from images of textures. Fig. 2 shows some experiments. Each experiment is displayed in one row, where the first image is the training image, and the other two images are generated by the learning algorithm.

Experiment 3: generating hybrid patterns. We can learn models (5) and (6) from images of mixed categories, and generate hybrid patterns. Figs. 3 and 4 display a few examples.

Experiment 4: generating scene patterns. We can learn the generative CNN model (19) with (18) from images of objects without requiring image alignment or object bounding boxes. The learning algorithm is based on the generative gradient (20). Fig 5 displays one example. In our experimental setting, we learn 10 filters at the layer Conv4, based on the VGG filters at the layer Conv3. The size of each Conv4 filter to be learned is $11 \times 11 \times 256$, and the stride of sub-sampling is 2. We run 4 parallel chains of Langevin dynamics for sampling and learning.

10. Conclusion

In this conceptual paper, we explore a generative perspective of CNN, and illustrate that it is possible to learn CNN units generatively. It can be interesting to scale up the learning of the general form of the generative CNN model (19) with multi-layer connection weights from images of non-aligned objects of multiple categories in unsupervised fashion.

Code and data

The code and data for experiments can be downloaded at <http://www.stat.ucla.edu/~yang.lu/project/deepFrame/main.html>

Acknowledgement

We thank Jifeng Dai for earlier collaboration on generative CNN. We thank Junhua Mao for sharing his expertise on

CNN. The work is supported by NSF DMS, ONR MURI, and DARPA.

References

- Dai, J.; Lu, Y.; and Wu, Y. N. 2015. Generative modeling of convolutional neural networks. In *ICLR*.
- Della Pietra, S.; Della Pietra, V.; and Lafferty, J. 1997. Inducing features of random fields. *Pattern Analysis and Machine Intelligence, IEEE Transactions on* 19(4):380–393.
- Deng, J.; Dong, W.; Socher, R.; Li, L.-J.; Li, K.; and Fei-Fei, L. 2009. Imagenet: A large-scale hierarchical image database. In *Computer Vision and Pattern Recognition, 2009. CVPR 2009. IEEE Conference on*, 248–255. IEEE.
- Denton, E.; Chintala, S.; Szlam, A.; and Fergus, R. 2015. Deep generative image models using a laplacian pyramid of adversarial networks. *ArXiv e-prints*.
- Dosovitskiy, E.; Springenberg, J. T.; and Brox, T. 2015. Learning to generate chairs with convolutional neural networks. In *IEEE International Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Gatys, L. A.; Ecker, A. S.; and Bethge, M. 2015. A neural algorithm of artistic style. *ArXiv e-prints*.
- Gregor, K.; Danihelka, I.; Graves, A.; Rezende, D. J.; and Wierstra, D. 2015. DRAW: A recurrent neural network for image generation. In *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015*, 1462–1471.
- Hinton, G.; Dayan, P.; Frey, B. J.; and Neal, R. M. 1995. The wake-sleep algorithm for unsupervised neural networks.
- Hinton, G.; Osindero, S.; Welling, M.; and Teh, Y.-W. 2006. Unsupervised discovery of nonlinear structure using contrastive backpropagation. *Cognitive science* 30(4):725–731.
- Hinton, G. E. 2002. Training products of experts by minimizing contrastive divergence. *Neural Computation* 14(8):1771–1800.
- Hong, Y.; Si, Z.; Hu, W.; Zhu, S.; and Wu, Y. 2014. Unsupervised learning of compositional sparse code for natural image representation. *Quarterly of Applied Mathematics* 79:373–406.

- Julesz, B. 1962. Visual pattern discrimination. *IRE Transactions on Information Theory* 8(2):84–92.
- Kingma, D. P., and Welling, M. 2014. Auto-encoding variational bayes. *ICLR*.
- Krizhevsky, A.; Sutskever, I.; and Hinton, G. E. 2012. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NIPS)*, 1097–1105.
- Kulkarni, T. D.; Whitney, W.; Kohli, P.; and Tenenbaum, J. B. 2015. Deep Convolutional Inverse Graphics Network. *ArXiv e-prints*.
- LeCun, Y.; Bottou, L.; Bengio, Y.; and Haffner, P. 1998. Gradient-based learning applied to document recognition. *Proceedings of the IEEE* 86(11):2278–2324.
- LeCun, Y.; Chopra, S.; Hadsell, R.; Ranzato, M.; and Huang, F. 2006. A tutorial on energy-based learning. In *Predicting Structured Data*. MIT Press.
- Mnih, A., and Gregor, K. 2014. Neural variational inference and learning in belief networks. In *Proceedings of the 31st International Conference on Machine Learning*.
- Neal, R. M. 2011. Mcmc using hamiltonian dynamics. *Handbook of Markov Chain Monte Carlo* 2.
- Ngiam, J.; Chen, Z.; Koh, P. W.; and Ng, A. Y. 2011. Learning deep energy models. In *International Conference on Machine Learning*.
- Rezende, D. J.; Mohamed, S.; and Wierstra, D. 2014. Stochastic backpropagation and approximate inference in deep generative models. In Jebara, T., and Xing, E. P., eds., *Proceedings of the 31st International Conference on Machine Learning (ICML-14)*, 1278–1286. JMLR Workshop and Conference Proceedings.
- Simonyan, K., and Zisserman, A. 2015. Very deep convolutional networks for large-scale image recognition. *ICLR*.
- Simonyan, K.; Vedaldi, A.; and Zisserman, A. 2015. Deep inside convolutional networks: Visualising image classification models and saliency maps. *ICLR*.
- Vedaldi, A., and Lenc, K. 2014. Matconvnet – convolutional neural networks for matlab. *CoRR* abs/1412.4564.
- Wu, Y. N.; Si, Z.; Gong, H.; and Zhu, S.-C. 2010. Learning active basis model for object detection and recognition. *International Journal of Computer Vision* 90:198–235.
- Wu, Y. N.; Zhu, S.-C.; and Liu, X. 2000. Equivalence of Julesz ensembles and frame models. *International Journal of Computer Vision* 38:247–265.
- Xie, J.; Hu, W.; Zhu, S.-C.; and Wu, Y. N. 2015a. Learning sparse frame models for natural image patterns. *International Journal of Computer Vision* 114:91–112.
- Xie, J.; Lu, Y.; Zhu, S.-C.; and Wu, Y. N. 2015b. Inducing wavelets into random fields via generative boosting. *Journal of Applied and Computational Harmonic Analysis*.
- Younes, L. 1999. On the convergence of markovian stochastic algorithms with rapidly decreasing ergodicity rates. *Stochastics: An International Journal of Probability and Stochastic Processes* 65(3-4):177–228.
- Zeiler, M. D., and Fergus, R. 2014. Visualizing and understanding convolutional neural networks. *ECCV*.
- Zhu, S. C., and Mumford, D. B. 1997. Prior learning and gibbs reaction-diffusion. *Pattern Analysis and Machine Intelligence, IEEE Transactions on* 19:1236–1250.
- Zhu, S. C., and Mumford, D. 2006. A stochastic grammar of images. *Foundations and Trends in Computer Graphics and Vision* 2(4):259–362.
- Zhu, S. C.; Liu, X.; and Wu, Y. N. 2000. Exploring texture ensembles by efficient markov chain monte carlo - towards a ‘trichromacy’ theory of texture. *Pattern Analysis and Machine Intelligence, IEEE Transactions on* 22:245–261.
- Zhu, S. C.; Wu, Y. N.; and Mumford, D. 1997. Minimax entropy principle and its application to texture modeling. *Neural Computation* 9(8):1627–1660.